MODULE 1

Q1. GitHub Repository Creation

Create a GitHub account and set up a new public repository named iot-summer-school-2026. Your repository must include:

- A README.md with your name, roll number, and a 3-line project description
- A folder structure: /week1, /week2, /week3, /final-project
- A .gitignore file configured for Arduino/C++ projects
- An appropriate open-source license (e.g., MIT)

Deliverable: Paste your GitHub repository URL in the submission form.

Url : anantpriyadarshi312-max/iot-summer-school-2026:

Q2. Git Workflow – Push Your First Code

Upload the following LED blink sketch to your repository under /week1/led_blink/:

```
void setup() {
  pinMode(13, OUTPUT);
}
void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

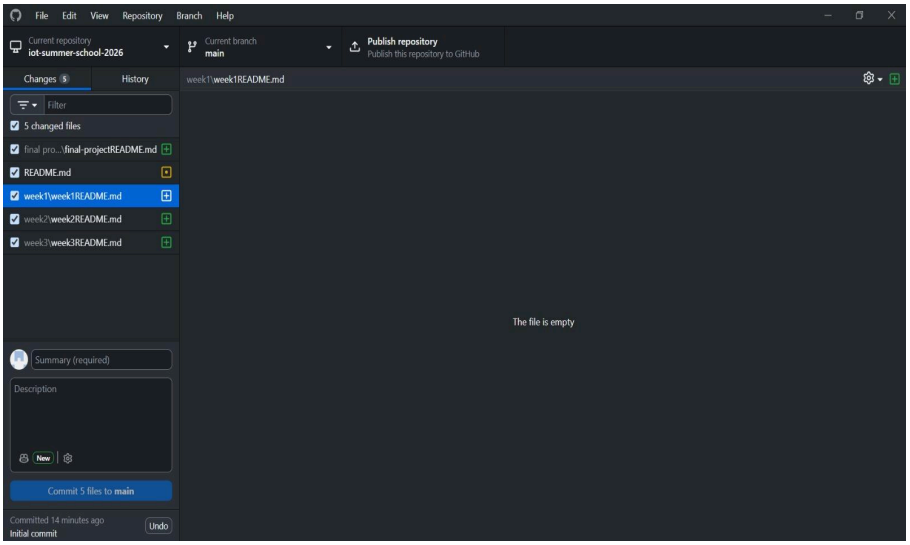
Commit this file with a meaningful commit message (e.g., feat: add LED blink starter code). Screenshot your commit history and upload it.

Q2. Git Workflow – Push Your First Code

Upload the following LED blink sketch to your repository under /week1/led_blink/:

```
void setup() {
  pinMode(13, OUTPUT);
}
void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

Commit this file with a meaningful commit message (e.g., feat: add LED blink starter code). Screenshot your commit history and upload it.

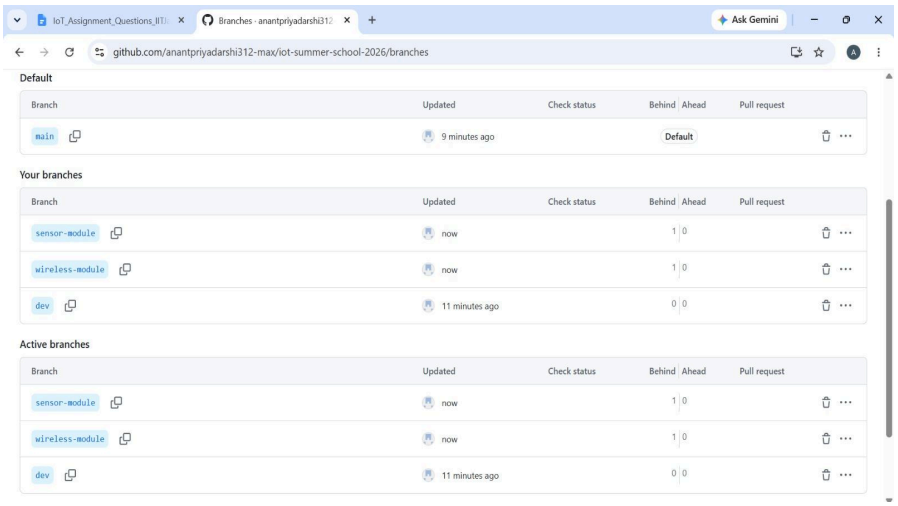


Q3. Branching Strategy

In your repository, create three branches named: dev, sensor-module, and wireless-module. In the dev branch, add a file notes.md describing what each branch will contain. Merge dev into main and show the merge commit in your submission.

Evaluation: GitHub branch graph screenshot + merge commit URL.

Url :
https://github.com/anantpriyadarshi312-max/iot-summer-school-2026/commit/97b9ff44b72a5d5582227f802238c010724106fa

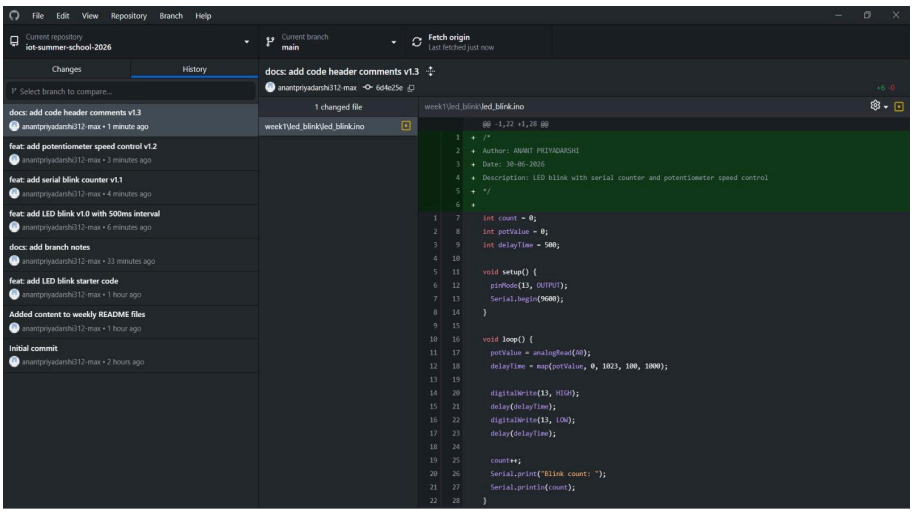


Q4. Versioned Code – Multiple Commits

Starting from your LED blink code (Q2), progressively improve it across 4 Git commits:

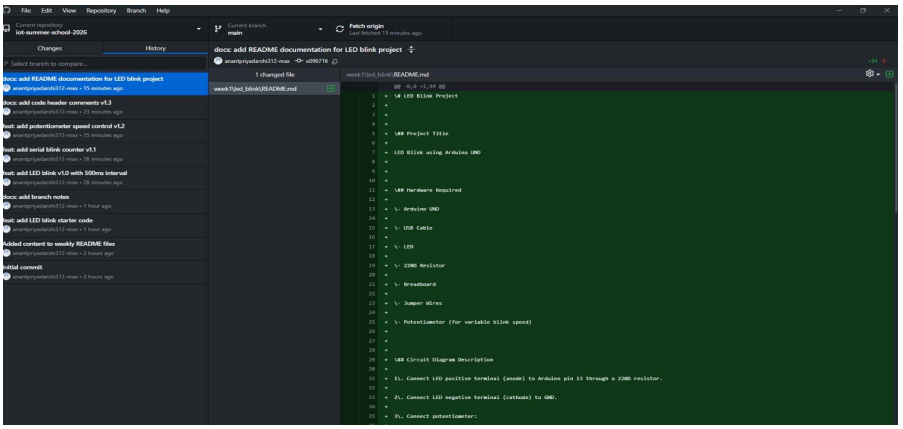
1. v1.0 – Basic LED blink (500ms interval)
2. v1.1 – Add serial print: Blink count: X
3. v1.2 – Read a potentiometer to control blink speed
4. v1.3 – Add a comment header block with author name, date, and description

Each commit must have a descriptive message following the conventional commits format (feat:, fix:, docs:, chore:).



Q5. README Documentation

Write a professional README.md for your /week1/led_blink/ project. The README must include: Project Title, Hardware Required (list of components), Circuit Diagram Description (text-based), How to Upload Code (step-by-step), Expected Output, and Troubleshooting Tips (minimum 3 points).

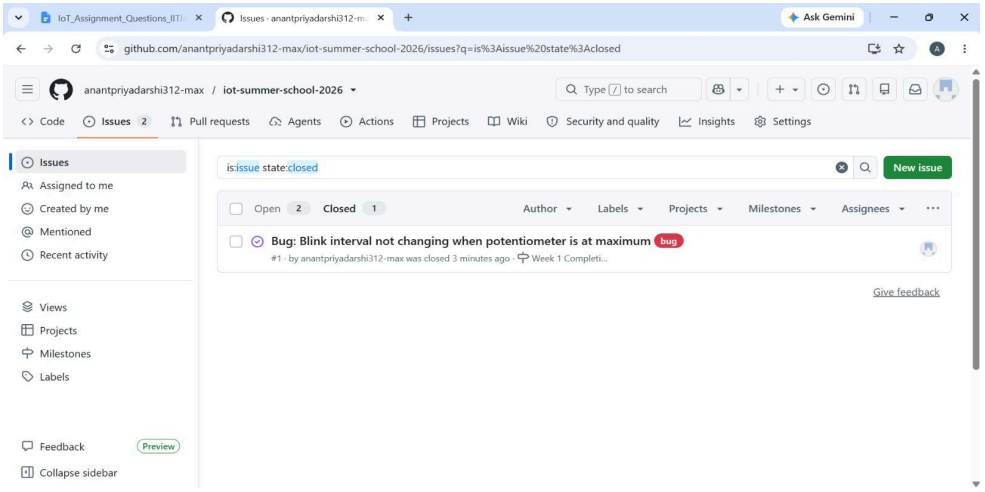


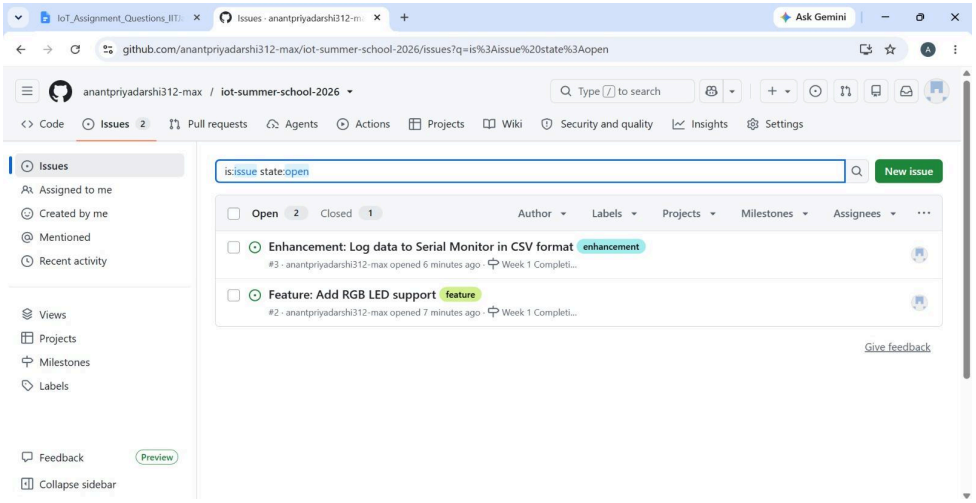
Q6. Issue Tracking & Milestones

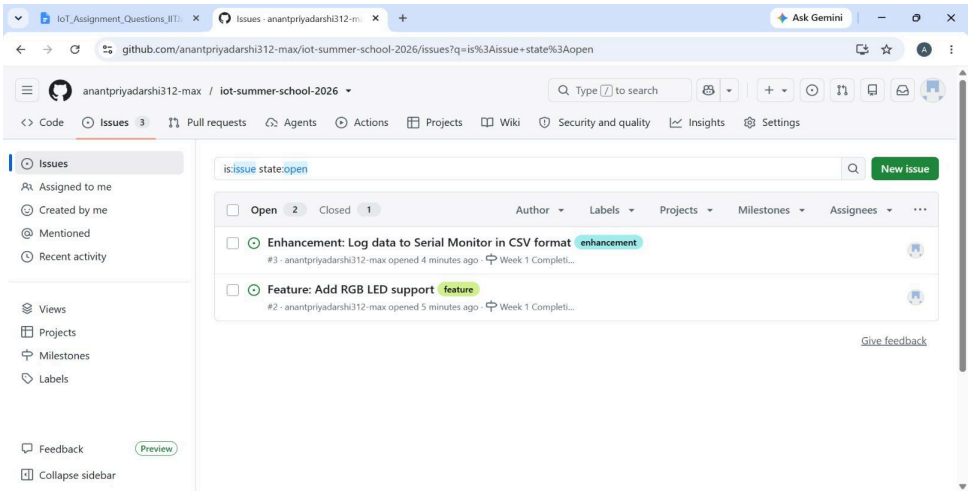
Create 3 GitHub Issues in your repository describing hypothetical bugs or features:

- Bug: Blink interval not changing when potentiometer is at maximum
- Feature: Add RGB LED support
- Enhancement: Log data to Serial Monitor in CSV format

Assign each issue to yourself, add appropriate labels, and create a milestone called 'Week 1 Completion'. Close at least one issue with a commit reference (Fixes #1).







Q9. Difference between `git pull`, `git fetch`, and `git clone`

`git clone`

Used to copy an entire remote repository to your local system for the first time.
Example:

git clone https://github.com/user/repo.git

Use when: Starting work on a project that is hosted remotely.

`git fetch`

Downloads latest changes from the remote repository but does **not merge** them into your current branch.
Example:

git fetch origin

Use when: You want to check updates before applying them.

`git pull`

Fetches changes from the remote repository **and merges** them into your current branch.
Example:

git pull origin main

Use when: You want to directly update your local branch with remote changes.

Q10. What is a `.gitignore` file?

A `.gitignore` file tells Git which files or folders should be ignored and not tracked in the repository. This is useful for excluding temporary files, build files, and system-generated files.

Example: compiled binaries, IDE settings, cache files.

`.gitignore` for Arduino Project

Compiled output files

*.hex

*.elf

OS-specific files

.DS_Store

Thumbs.db

IDE / build folders

.vscode/

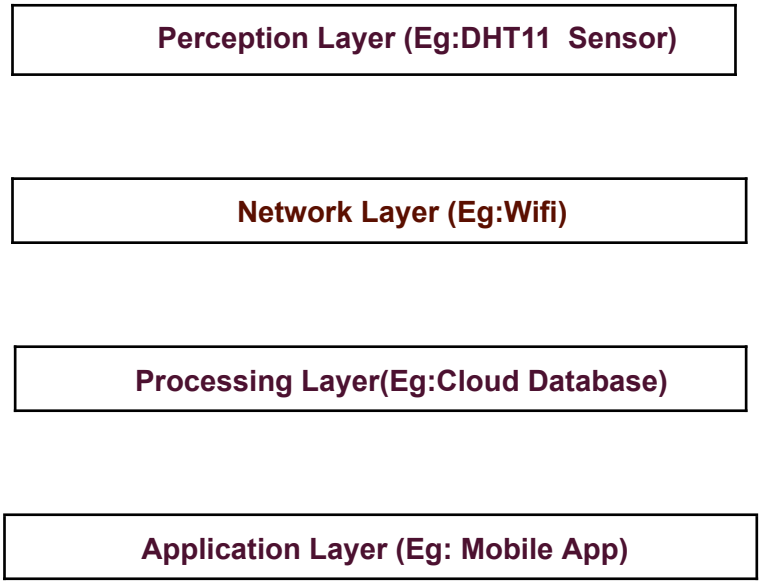
build/

Use: Prevents unnecessary files from being uploaded to GitHub.

MODULE 2

Q11. IoT Architecture

Draw and label a 4-layer IoT architecture diagram: Perception Layer, Network Layer, Processing Layer, and Application Layer. Give one example device/technology for each layer. (Hand-drawn photo accepted; upload to GitHub under /theory/)



Q12. Microcontroller vs Microprocessor

Fill in the following comparison table and upload a photo/scan to your GitHub repository:

Parameter	Microcontroller (Arduino UNO)	Microprocessor (Raspberry Pi)
CPU Speed	16 MHz	1.2–2.4 GHz (depends on model)
On-chip RAM	2 KB SRAM	No significant on-chip RAM; uses external RAM (1–8 GB)
On-chip Flash	32 KB Flash	No on-chip flash; uses SD card storage
Operating System	No OS / Bare-metal programming	Runs OS like Raspberry Pi OS or Linux
Typical Use Case	Sensor reading, embedded control, automation	Image processing, AI, web server, complex computing
Power Consumption	Very low (~0.2–0.5 W)	Higher (~3–15 W)

Q13. Arduino Pin Types

List and explain with examples: Digital Input, Digital Output, Analog Input, PWM Output, and I2C/SPI pins available on Arduino UNO. For each, give one real IoT use case.

1. Digital Input Pins

- Used to read HIGH (1) or LOW (0) signals.
- Pins: D0–D13 can be used as digital input.

Example:

Push button input.

```
int button = 2;


void setup() {
 pinMode(button, INPUT);
}
```

IoT Use Case:

Smart doorbell system → Detect button press and send notification.

---

2. Digital Output Pins

- Used to send HIGH (5V) or LOW (0V) signals.
- Pins: D0–D13

Example:

LED control.

```
int led = 13;


void setup() {
 pinMode(led, OUTPUT);
}


void loop() {
 digitalWrite(led, HIGH);
}
```

IoT Use Case:

Smart home → Turn light ON/OFF remotely.

---

3. Analog Input Pins

- Reads analog voltage values (0–5V).
- Converted into digital values (0–1023) using ADC.
- Pins: A0–A5

Example:

Temperature sensor or potentiometer.

```
int sensor = A0;
```

```
int value;


void loop() {

 value = analogRead(sensor);

}
```

IoT Use Case:

Weather monitoring system → Read temperature/humidity sensors.

---

4. PWM Output Pins

- PWM = Pulse Width Modulation.
- Simulates analog output by varying duty cycle.
- Pins marked with ~: 3, 5, 6, 9, 10, 11

Example:

Control motor speed or LED brightness.

```
analogWrite(9, 128);
```

(128 ≈ 50% duty cycle)

IoT Use Case:

Smart fan system → Control fan speed automatically.

---

5. I2C Pins

- Used for communication between multiple devices using 2 wires.
- Pins:
    - SDA = A4
    - SCL = A5

Example Devices:

- OLED display
- DHT11 (typically single-wire, but common sensor example)
- RTC module

IoT Use Case:

Smart monitoring → Display sensor data on OLED.

---

6. SPI Pins

- High-speed communication protocol.
- Pins:
    - MOSI = D11
    - MISO = D12
    - SCK = D13
    - SS = D10

Example Devices:

- SD card module
- Ethernet module

IoT Use Case:

Data logging system → Store sensor data in SD card.

Q14. Traffic Light Controller

Write an Arduino program that simulates a traffic light system using 3 LEDs (Red, Yellow, Green):

- RED ON for 5 seconds → YELLOW ON for 2 seconds → GREEN ON for 4 seconds
- Add a pedestrian button (push button on pin 7): when pressed, force RED immediately and hold for 8 seconds
- Print the current light state to Serial Monitor with timestamp (millis())

Code must be uploaded to: /week2/traffic_light/traffic_light.ino

Repo Link:

iot-summer-school-2026/week2/traffic_light/traffic_light.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
const int redLED = 2;

const int yellowLED = 3;

const int greenLED = 4;
```

```
const int buttonPin = 7;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);

  Serial.begin(9600);
}

void setLight(bool red, bool yellow, bool green, String state) {
  digitalWrite(redLED, red);
  digitalWrite(yellowLED, yellow);
  digitalWrite(greenLED, green);

  Serial.print("Time: ");
  Serial.print(millis());
  Serial.print(" ms | State: ");
  Serial.println(state);
}

void loop() {

  // Check pedestrian button
  if (digitalRead(buttonPin) == LOW) {
```

```
  setLight(HIGH, LOW, LOW, "PEDESTRIAN - RED");
  delay(8000);
  return;
}

// RED ON for 5 sec
setLight(HIGH, LOW, LOW, "RED");
for (int i = 0; i < 50; i++) {
  if (digitalRead(buttonPin) == LOW) {
    setLight(HIGH, LOW, LOW, "PEDESTRIAN - RED");
    delay(8000);
    return;
  }
  delay(100);
}

// YELLOW ON for 2 sec
setLight(LOW, HIGH, LOW, "YELLOW");
for (int i = 0; i < 20; i++) {
  if (digitalRead(buttonPin) == LOW) {
    setLight(HIGH, LOW, LOW, "PEDESTRIAN - RED");
    delay(8000);
    return;
  }
  delay(100);
}
```

```
  // GREEN ON for 4 sec
  setLight(LOW, LOW, HIGH, "GREEN");
  for (int i = 0; i < 40; i++) {
    if (digitalRead(buttonPin) == LOW) {
      setLight(HIGH, LOW, LOW, "PEDESTRIAN - RED");
      delay(8000);
      return;
    }
    delay(100);
  }
}
```

Q15. Digital Piano using Buzzer

Build a 4-key musical instrument using push buttons and a passive buzzer:

- 4 buttons mapped to notes: Do (262Hz), Re (294Hz), Mi (330Hz), Fa (349Hz)
- Button press plays the note; button release silences the buzzer
- If two buttons are pressed together: play Sol (392Hz) as a chord substitute
- Add a mode toggle (5th button): switch between Major and Minor scale

Commit separately for each feature using Git.

Link:

iot-summer-school-2026/week2/digital_piano/digital_piano.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code:

Buzzer + Button Setup

```
const int buzzer = 8;
const int buttons[4] = {2, 3, 4, 5};

void setup() {
  for (int i = 0; i < 4; i++) {
    pinMode(buttons[i], INPUT_PULLUP);
  }
  pinMode(buzzer, OUTPUT);
}
void loop() {
}
```

4 key Piano

```
const int buzzer = 8;
const int buttons[4] = {2, 3, 4, 5};
int notes[4] = {262, 294, 330, 349};

void setup() {
  for (int i = 0; i < 4; i++) {
    pinMode(buttons[i], INPUT_PULLUP);
  }
  pinMode(buzzer, OUTPUT);
}

void loop() {
  bool played = false;
```

```
  for (int i = 0; i < 4; i++) {
    if (digitalRead(buttons[i]) == LOW) {
      tone(buzzer, notes[i]);
      played = true;
      break;
    }
  }

  if (!played) {
    noTone(buzzer);
  }
}
```

Two Buttons

```
void loop() {
  int count = 0;
  int lastPressed = -1;

  for (int i = 0; i < 4; i++) {
    if (digitalRead(buttons[i]) == LOW) {
      count++;
      lastPressed = i;
    }
  }

  if (count == 0) {
    noTone(buzzer);
```

```
  }
  else if (count >= 2) {
    tone(buzzer, 392); // Sol
  }
  else {
    tone(buzzer, notes[lastPressed]);
  }
}
```

Major/Minor Mode Toggle

```
const int buzzer = 8;
const int buttons[4] = {2, 3, 4, 5};
const int modeButton = 6;

bool majorMode = true;
bool lastModeState = HIGH;

int majorNotes[4] = {262, 294, 330, 349}; // Do Re Mi Fa
int minorNotes[4] = {262, 294, 311, 349}; // Minor version

void setup() {
  for (int i = 0; i < 4; i++) {
    pinMode(buttons[i], INPUT_PULLUP);
  }

  pinMode(modeButton, INPUT_PULLUP);
  pinMode(buzzer, OUTPUT);
```

```
}

void loop() {
  // Mode toggle
  bool modeState = digitalRead(modeButton);

  if (modeState == LOW && lastModeState == HIGH) {
    majorMode = !majorMode;
    delay(200); // debounce
  }
  lastModeState = modeState;

  int count = 0;
  int lastPressed = -1;

  // Check pressed buttons
  for (int i = 0; i < 4; i++) {
    if (digitalRead(buttons[i]) == LOW) {
      count++;
      lastPressed = i;
    }
  }

  if (count == 0) {
    noTone(buzzer);
  }
```

```
  else if (count >= 2) {
    tone(buzzer, 392); // Sol chord substitute
  }
  else {
    if (majorMode) {
      tone(buzzer, majorNotes[lastPressed]);
    } else {
      tone(buzzer, minorNotes[lastPressed]);
    }
  }
}
```

Q16. Serial Command Interface

Write a program that receives commands over Serial Monitor and controls hardware:

```
LED_ON  → Turn built-in LED on
LED_OFF → Turn built-in LED off
BLINK_X → Blink LED X times (X is 1-9)
STATUS  → Print pin states
RESET   → Restart blink counter
```

Implement input validation: unknown commands should print 'ERROR: Unknown command'.

Link

[iot-summer-school-2026/week2/serial_command/serial_command.ino at main · anantpriyadarshi312-max/iot-summer-school-2026]

Code

```
int blinkCounter = 0;
bool ledState = false;

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(9600);
  Serial.println("Serial Command Interface Ready");
}
```

```
void loop() {
  if (Serial.available()) {
    String command = Serial.readStringUntil('\n');
    command.trim();

    if (command == "LED_ON") {
      digitalWrite(LED_BUILTIN, HIGH);
      ledState = true;
      Serial.println("LED turned ON");
    }

    else if (command == "LED_OFF") {
      digitalWrite(LED_BUILTIN, LOW);
      ledState = false;
      Serial.println("LED turned OFF");
    }

    else if (command.startsWith("BLINK_")) {
      String numStr = command.substring(6);
      int count = numStr.toInt();

      if (count >= 1 && count <= 9) {
        for (int i = 0; i < count; i++) {
          digitalWrite(LED_BUILTIN, HIGH);
          delay(500);
          digitalWrite(LED_BUILTIN, LOW);
          delay(500);
        }
        blinkCounter += count;
        ledState = false;
        Serial.print("Blinked ");
        Serial.print(count);
        Serial.println(" times");
      } else {
        Serial.println("ERROR: Invalid blink count");
      }
```

```
    }

    else if (command == "STATUS") {
      Serial.print("LED State: ");
      Serial.println(ledState ? "ON" : "OFF");

      Serial.print("Blink Counter: ");
      Serial.println(blinkCounter);
    }

    else if (command == "RESET") {
      blinkCounter = 0;
      Serial.println("Blink counter reset");
    }

    else {
      Serial.println("ERROR: Unknown command");
    }
  }
}
```

Q17. PWM Fading Night Light

Create an automatic night light that fades an LED in and out (breathing effect) continuously. Implement three modes switchable via a button: (1) Slow breathing (3-second cycle), (2) Fast pulse (0.5-second cycle), (3) SOS pattern (... --- ...). Display current mode on Serial Monitor.

Link

iot-summer-school-2026/week2/night_light/night_light.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
const int ledPin = 9;
const int buttonPin = 2;

int mode = 1;
bool lastButtonState = HIGH;
```

```
void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);

  Serial.begin(9600);
  Serial.println("Mode 1: Slow Breathing");
}

void loop() {
  bool buttonState = digitalRead(buttonPin);

  if (buttonState == LOW && lastButtonState == HIGH) {
    mode++;
    if (mode > 3) mode = 1;

    if (mode == 1) Serial.println("Mode 1: Slow Breathing");
    if (mode == 2) Serial.println("Mode 2: Fast Pulse");
    if (mode == 3) Serial.println("Mode 3: SOS Pattern");

    delay(200);
  }

  lastButtonState = buttonState;

  if (mode == 1) slowBreathing();
  else if (mode == 2) fastPulse();
  else if (mode == 3) sosPattern();
}

void slowBreathing() {
  for (int i = 0; i <= 255; i++) {
    analogWrite(ledPin, i);
    delay(6);
  }

  for (int i = 255; i >= 0; i--) {
    analogWrite(ledPin, i);
```

```
    delay(6);
  }
}

void fastPulse() {
  analogWrite(ledPin, 255);
  delay(250);
  analogWrite(ledPin, 0);
  delay(250);
}

void sosPattern() {
  blinkShort();
  blinkShort();
  blinkShort();

  blinkLong();
  blinkLong();
  blinkLong();

  blinkShort();
  blinkShort();
  blinkShort();

  delay(500);
}

void blinkShort() {
  analogWrite(ledPin, 255);
  delay(200);
  analogWrite(ledPin, 0);
  delay(200);
}

void blinkLong() {
  analogWrite(ledPin, 255);
  delay(600);
```

```
  analogWrite(ledPin, 0);
  delay(200);
}
```

Q18. State Machine Implementation

Implement a simple vending machine state machine with 4 states: IDLE, COIN_INSERTED, ITEM_SELECTED, DISPENSING. Use 3 buttons as inputs (Insert Coin, Select Item, Cancel). Use 3 LEDs to show the current state. Print all state transitions to Serial Monitor. Code must include a comment block documenting the state transition diagram.

Link

[iot-summer-school-2026/week2/vending_machine/vending_machine.ino at main · anantpriyadarshi312-max/iot-summer-school-2026](#) \

Code

```
/*
=======================================================
VENDING MACHINE STATE MACHINE
=======================================================

States:
1. IDLE
2. COIN_INSERTED
3. ITEM_SELECTED
4. DISPENSING

State Transition Diagram

            Insert Coin
IDLE ------------------------> COIN_INSERTED
 ^                              |
 | Cancel                       | Select Item
 |                              v
 +--------------------------- ITEM_SELECTED
                                |
                                | Dispense
                                v
```

```
                    DISPENSING
                        |
                        | Finished
                        v
                      IDLE
```

Cancel can return to IDLE from
COIN_INSERTED or ITEM_SELECTED.

```
==================================================
*/

const int coinButton   = 2;
const int selectButton = 3;
const int cancelButton = 4;

const int redLED    = 8;
const int yellowLED = 9;
const int greenLED  = 10;

enum State {
  IDLE,
  COIN_INSERTED,
  ITEM_SELECTED,
  DISPENSING
};

State currentState = IDLE;

void setup() {

  pinMode(coinButton, INPUT_PULLUP);
  pinMode(selectButton, INPUT_PULLUP);
  pinMode(cancelButton, INPUT_PULLUP);

  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
```

```
  pinMode(greenLED, OUTPUT);

  Serial.begin(9600);

  updateLEDs();
  Serial.println("Current State : IDLE");
}

void loop() {

  switch (currentState) {

    case IDLE:

      if (digitalRead(coinButton) == LOW) {
        currentState = COIN_INSERTED;
        Serial.println("Transition: IDLE -> COIN_INSERTED");
        updateLEDs();
        delay(300);
      }

      break;

    case COIN_INSERTED:

      if (digitalRead(selectButton) == LOW) {
        currentState = ITEM_SELECTED;
        Serial.println("Transition: COIN_INSERTED -> ITEM_SELECTED");
        updateLEDs();
        delay(300);
      }

      else if (digitalRead(cancelButton) == LOW) {
        currentState = IDLE;
        Serial.println("Transition: COIN_INSERTED -> IDLE (Cancel)");
        updateLEDs();
        delay(300);
```

```
      }

      break;

    case ITEM_SELECTED:

      if (digitalRead(cancelButton) == LOW) {
        currentState = IDLE;
        Serial.println("Transition: ITEM_SELECTED -> IDLE (Cancel)");
        updateLEDs();
        delay(300);
      }

      else {

        delay(1000);

        currentState = DISPENSING;
        Serial.println("Transition: ITEM_SELECTED -> DISPENSING");
        updateLEDs();

        delay(3000);

        currentState = IDLE;
        Serial.println("Transition: DISPENSING -> IDLE");
        updateLEDs();
      }

      break;

    case DISPENSING:
      break;
  }
}

void updateLEDs() {
```

```
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, LOW);

  switch (currentState) {

    case IDLE:
      digitalWrite(redLED, HIGH);
      break;

    case COIN_INSERTED:
      digitalWrite(yellowLED, HIGH);
      break;

    case ITEM_SELECTED:
      digitalWrite(greenLED, HIGH);
      break;

    case DISPENSING:
      digitalWrite(redLED, HIGH);
      digitalWrite(yellowLED, HIGH);
      digitalWrite(greenLED, HIGH);
      break;
  }
}
```

Q19. Difference between `analogWrite()` and `analogRead()`. What is PWM?

Difference between `analogRead()` and `analogWrite()`

Feature	`analogRead()`	`analogWrite()`
Purpose	Reads an analog voltage from a sensor	Generates a PWM signal to simulate analog output
Pins Used	Analog pins (A0–A5 on Arduino UNO)	PWM pins (3, 5, 6, 9, 10, 11 on Arduino UNO)

Input/Output	Input	Output
Value Range	0–1023	0–255
Example	int value = analogRead(A0);	analogWrite(9, 128);

What is PWM?

PWM (Pulse Width Modulation) is a technique that varies the **duty cycle** of a digital signal to produce the effect of different analog output levels. Although the output switches between HIGH and LOW, changing the percentage of time it stays HIGH controls the average power delivered to a device.

Why is PWM used?

- To control LED brightness.
- To control DC motor speed.
- To control fan speed.
- To generate simple analog-like output without a true DAC.

Practical IoT Examples

- `analogRead()` **example:** A smart weather station reads data from a light sensor or soil moisture sensor to monitor environmental conditions.
- `analogWrite()` **example:** A smart street-light system adjusts LED brightness automatically at night using PWM.

Q20. Explain `setup()` and `loop()` in Arduino. What is the effect of a long `delay()`? What is the non-blocking alternative?

`setup()`

- Runs **only once** when the Arduino is powered on or reset.
- Used for initialization such as:
  - Setting pin modes (`pinMode()`)
  - Starting serial communication (`Serial.begin()`)
  - Initializing sensors and modules

Example:

```
void setup() {
 pinMode(13, OUTPUT);
 Serial.begin(9600);
}
```

`loop()`

- Runs **continuously** after `setup()` finishes.
- Contains the main program logic that repeats forever.

Example:

```
void loop() {
 digitalWrite(13, HIGH);
 delay(1000);
 digitalWrite(13, LOW);
 delay(1000);
}
```

What happens if you put a long `delay()` inside `loop()`?

A long `delay()` **blocks the processor**, so the Arduino cannot perform any other tasks during that time.

Effects:

- Slow response to button presses.
- Sensor readings are delayed.
- Serial communication is not processed until the delay ends.
- Overall responsiveness of the system decreases.

For example, if `delay(5000)` is used, the Arduino waits 5 seconds before checking sensors or inputs again.

How does it affect sensor reading responsiveness?

Sensor values are updated only after the delay finishes. If a sensor changes state during the delay, the Arduino will not detect it immediately, making the system less responsive.

What is the non-blocking alternative?

The recommended non-blocking alternative is to use the **`millis()`** function.

- `millis()` returns the number of milliseconds since the Arduino started running.
- It allows tasks to be performed at specific time intervals **without stopping the program**, so sensors, buttons, and communication can still be checked continuously.

Example use case: A smart irrigation system can read soil moisture every second while simultaneously monitoring a button press and updating an LCD display, all without blocking execution.

MODULE 3

Q21. Environmental Monitoring Station (DHT11)

Build a mini weather station using DHT11 sensor:

- Read temperature (°C and °F) and humidity every 2 seconds
- Display readings on Serial Monitor in CSV format: timestamp,temp_C,temp_F,humidity
- Trigger a Red LED if temperature > 35°C or humidity > 80%
- Trigger a Green LED if conditions are normal
- Save 5 sample CSV readings in a file /week3/data/sample_readings.csv in your GitHub repo

Library: DHTesp or DHT by Adafruit. Include library version in README.

Link :
iot-summer-school-2026/week3/environmental_monitoring/environmental_monitoring.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code :

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

#define RED_LED 8
#define GREEN_LED 9

DHT dht(DHTPIN, DHTTYPE);

void setup() {
```

```
  Serial.begin(9600);

  dht.begin();

  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);

  // CSV Header
  Serial.println("timestamp,temp_C,temp_F,humidity");
}

void loop() {

  float tempC = dht.readTemperature();
  float humidity = dht.readHumidity();
  float tempF = dht.readTemperature(true);

  if (isnan(tempC) || isnan(tempF) || isnan(humidity)) {
    Serial.println("Sensor Read Error");
    delay(2000);
    return;
  }

  // Print CSV format
  Serial.print(millis());
  Serial.print(",");

  Serial.print(tempC);
  Serial.print(",");

  Serial.print(tempF);
  Serial.print(",");

  Serial.println(humidity);

  // LED Status
```

```
  if (tempC > 35 || humidity > 80) {

    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);

  } else {

    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);

  }

  delay(2000);
}
```

Q22. Ultrasonic Parking Sensor

Using HC-SR04, build a parking distance alert system:

- Distance > 50cm: No alert, Serial prints SAFE
- Distance 20–50cm: Yellow LED ON, buzzer beeps every 500ms
- Distance 10–20cm: Red LED ON, buzzer beeps every 200ms
- Distance < 10cm: All LEDs flash rapidly, buzzer continuous
- Calculate and display distance in cm using the formula: distance = (duration * 0.034) / 2

Advanced: Use millis() for non-blocking timing (no delay()). +1 bonus mark.

Link:

iot-summer-school-2026/week3/parking_sensor/parking_sensor.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
#include <DHT.h>
#define DHTPIN 2
#define DHTTYPE DHT11
#define RED_LED 8
#define GREEN_LED 9
DHT dht(DHTPIN, DHTTYPE);
void setup() {
```

```
  Serial.begin(9600);
  dht.begin();
  pinMode(RED_LED, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  // CSV Header
  Serial.println("timestamp,temp_C,temp_F,humidity");
}
void loop() {
  float tempC = dht.readTemperature();
  float humidity = dht.readHumidity();
  float tempF = dht.readTemperature(true);
  if (isnan(tempC) || isnan(tempF) || isnan(humidity)) {
    Serial.println("Sensor Read Error");
    delay(2000);
    return;
  }
  // Print CSV format
  Serial.print(millis());
  Serial.print(",");
  Serial.print(tempC);
  Serial.print(",");
  Serial.print(tempF);
  Serial.print(",");
  Serial.println(humidity);
  // LED Status
  if (tempC > 35 || humidity > 80) {

    digitalWrite(RED_LED, HIGH);
    digitalWrite(GREEN_LED, LOW);
  } else {
    digitalWrite(RED_LED, LOW);
    digitalWrite(GREEN_LED, HIGH);
  }
  delay(2000);
}
```

Q23. Smart Street Light (LDR + PIR)

Implement an intelligent street light controller:

- LDR below threshold (dark): Enable motion detection mode
- PIR detects motion at night: Turn LED on at full brightness for 30 seconds
- No motion after 30 sec: Dim LED to 20% using PWM
- LDR above threshold (daylight): Turn LED completely OFF
- Log every event to Serial: [HH:MM:SS] EVENT: description

Submit a circuit diagram sketch (hand-drawn or Tinkercad screenshot) in /week3/schematics/.

Link

iot-summer-school-2026/week3/schematics/street_light_schematic.jpg at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
/*
Smart Street Light
LDR + PIR Sensor
*/

const int ldrPin = A0;
const int pirPin = 2;
const int ledPin = 9;

const int LDR_THRESHOLD = 500;

unsigned long motionTime = 0;
bool motionDetected = false;

void printTime()
{
  unsigned long seconds = millis() / 1000;

  int hh = seconds / 3600;
  int mm = (seconds % 3600) / 60;
  int ss = seconds % 60;

  if(hh < 10) Serial.print("0");
  Serial.print(hh);
  Serial.print(":");
```

```
  if(mm < 10) Serial.print("0");
  Serial.print(mm);
  Serial.print(":");

  if(ss < 10) Serial.print("0");
  Serial.print(ss);
}

void logEvent(String msg)
{
  Serial.print("[");
  printTime();
  Serial.print("] EVENT: ");
  Serial.println(msg);
}

void setup()
{
  pinMode(pirPin, INPUT);
  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);

  logEvent("System Started");
}

void loop()
{
  int ldrValue = analogRead(ldrPin);

  // DAY
  if(ldrValue > LDR_THRESHOLD)
  {
    analogWrite(ledPin,0);

    motionDetected = false;
```

```
    logEvent("Daylight - LED OFF");

    delay(1000);

    return;
  }

  // NIGHT

  if(digitalRead(pirPin) == HIGH)
  {
    analogWrite(ledPin,255);

    motionDetected = true;

    motionTime = millis();

    logEvent("Motion Detected - LED FULL BRIGHTNESS");
  }

  if(motionDetected)
  {
    if(millis() - motionTime >= 30000)
    {
      analogWrite(ledPin,51);

      motionDetected = false;

      logEvent("No Motion - LED DIM (20%)");
    }
  }
  else
  {
    analogWrite(ledPin,51);
  }
}
```

Q24. Multi-Sensor Data Logger

Build a system that reads 3 sensors simultaneously and logs structured data:

- DHT11: Temperature, Humidity
- LDR: Light level (0–1023 raw, converted to percentage)
- HC-SR04: Distance in cm

Output format every 5 seconds:

```
=== SENSOR LOG ===
Time       : 12345 ms
Temp       : 28.5 C | Humidity: 65%
Light      : 73% (Bright)
Distance   : 42 cm
==================
```

Upload 20 lines of captured Serial Monitor output as /week3/data/sensor_log.txt.

Link

iot-summer-school-2026/week3/multi_sensor_logger/data.txt at main · anantpriyadarshi312-max/iot-summer-school-2026

Q25. Servo Motor Sweep

Control a SG90 servo motor with a potentiometer. Potentiometer position (0–1023) maps to servo angle (0°–180°). Display angle in Serial Monitor. Add a button that makes the servo do one full sweep (0→180→0) when pressed. Code: /week3/servo_control/

Link:

iot-summer-school-2026/week3/servo_control/servo_control.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
#include <Servo.h>

Servo myServo;

const int servoPin = 9;
const int potPin = A0;
const int buttonPin = 2;

int potValue;
int angle;
```

```
void setup()
{
  Serial.begin(9600);

  myServo.attach(servoPin);

  pinMode(buttonPin, INPUT_PULLUP);
}

void loop()
{
  // If button is pressed
  if (digitalRead(buttonPin) == LOW)
  {
    // Sweep from 0 to 180
    for(int pos = 0; pos <= 180; pos++)
    {
      myServo.write(pos);
      Serial.print("Sweep Angle: ");
      Serial.println(pos);
      delay(15);
    }

    // Sweep back from 180 to 0
    for(int pos = 180; pos >= 0; pos--)
    {
      myServo.write(pos);
      Serial.print("Sweep Angle: ");
      Serial.println(pos);
      delay(15);
    }

    delay(300);
  }

  // Manual control using potentiometer
  else
```

```
 {
   potValue = analogRead(potPin);

   angle = map(potValue,0,1023,0,180);

   myServo.write(angle);

   Serial.print("Pot Value: ");
   Serial.print(potValue);

   Serial.print("    Servo Angle: ");
   Serial.println(angle);

   delay(100);
 }
}
```

Q26. DC Motor Speed Control with L298N

Control a DC motor through L298N driver: potentiometer sets speed (0–255 PWM), one button changes direction (Forward/Reverse), another button stops/starts the motor. Display Direction, Speed (0–100%), and State on Serial Monitor.

Link:

iot-summer-school-2026/week3/dc_motor/dc_motor.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
#include <Servo.h>

Servo myServo;

const int servoPin = 9;
const int potPin = A0;
const int buttonPin = 2;

int potValue;
int angle;

void setup()
```

```
{
  Serial.begin(9600);

  myServo.attach(servoPin);

  pinMode(buttonPin, INPUT_PULLUP);
}

void loop()
{
  // If button is pressed
  if (digitalRead(buttonPin) == LOW)
  {
    // Sweep from 0 to 180
    for(int pos = 0; pos <= 180; pos++)
    {
      myServo.write(pos);
      Serial.print("Sweep Angle: ");
      Serial.println(pos);
      delay(15);
    }

    // Sweep back from 180 to 0
    for(int pos = 180; pos >= 0; pos--)
    {
      myServo.write(pos);
      Serial.print("Sweep Angle: ");
      Serial.println(pos);
      delay(15);
    }

    delay(300);
  }

  // Manual control using potentiometer
  else
  {
```

```
    potValue = analogRead(potPin);

    angle = map(potValue,0,1023,0,180);

    myServo.write(angle);

    Serial.print("Pot Value: ");
    Serial.print(potValue);

    Serial.print("    Servo Angle: ");
    Serial.println(angle);

    delay(100);
  }
}
```

Q27. Relay-Controlled AC Device Simulation

Simulate a relay controlling an AC appliance (use LED as a safe simulation). DHT11 triggers relay ON when temperature > 32°C (AC turns on). Relay OFF when temp drops below 28°C (hysteresis). Add a manual override button. Log all relay state changes with temperature readings.

Link

iot-summer-school-2026/week3/relay_control/relay_control.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11

#define RELAY_PIN 8
#define BUTTON_PIN 3

DHT dht(DHTPIN, DHTTYPE);

bool relayState = false;
bool manualOverride = false;
```

```
int lastButtonState = HIGH;

void setup()
{
  Serial.begin(9600);

  dht.begin();

  pinMode(RELAY_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  digitalWrite(RELAY_PIN, LOW);
}

void loop()
{
  // ---------- Read Button ----------
  int buttonState = digitalRead(BUTTON_PIN);

  if(buttonState == LOW && lastButtonState == HIGH)
  {
    manualOverride = !manualOverride;

    if(manualOverride)
      relayState = !relayState;

    Serial.println("------------------------");

    if(manualOverride)
      Serial.println("Manual Override ENABLED");
    else
      Serial.println("Manual Override DISABLED");

    delay(250);
  }

  lastButtonState = buttonState;
```

```
// ---------- Read Temperature ----------
float temp = dht.readTemperature();

if(isnan(temp))
{
  Serial.println("DHT Error");
  delay(1000);
  return;
}

bool previousState = relayState;

// ---------- Automatic Control ----------
if(!manualOverride)
{
  if(temp > 32)
    relayState = true;

  else if(temp < 28)
    relayState = false;
}

digitalWrite(RELAY_PIN, relayState);

// ---------- Log only when state changes ----------
if(previousState != relayState)
{
  Serial.println("------------------------");
  Serial.print("Temperature : ");
  Serial.print(temp);
  Serial.println(" C");

  Serial.print("Relay : ");

  if(relayState)
    Serial.println("ON (AC ON)");
```

```
    else
      Serial.println("OFF (AC OFF)");
  }

  delay(1000);
}
```

Q28. Keypad + LCD Display

Using a 4x4 membrane keypad and 16x2 LCD, build a password-protected access system: Display 'ENTER PIN:' on LCD, user enters 4-digit PIN, correct PIN (set in code) shows 'ACCESS GRANTED' + Green LED, wrong PIN shows 'ACCESS DENIED' + Red LED + buzzer. After 3 wrong attempts, lock for 10 seconds.

Link

iot-summer-school-2026/week3/keypad/keypad.ino at main · anantpriyadarshi312-max/iot-summer-school-2026

Code

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>

LiquidCrystal_I2C lcd(0x27,16,2);

// Keypad
const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS]={
{'1','2','3','A'},
{'4','5','6','B'},
{'7','8','9','C'},
{'*','0','#','D'}
};

byte rowPins[ROWS]={2,3,4,5};
byte colPins[COLS]={6,7,8,9};

Keypad keypad=Keypad(makeKeymap(keys),rowPins,colPins,ROWS,COLS);

const int greenLED=10;
```

```
const int redLED=11;
const int buzzer=12;

String password="1234";
String input="";

int attempts=0;

void setup()
{
  lcd.init();
  lcd.backlight();

  pinMode(greenLED,OUTPUT);
  pinMode(redLED,OUTPUT);
  pinMode(buzzer,OUTPUT);

  lcd.setCursor(0,0);
  lcd.print("ENTER PIN:");
}

void loop()
{
  char key=keypad.getKey();

  if(key)
  {
    if(key>='0' && key<='9')
    {
      input+=key;

      lcd.setCursor(input.length()-1,1);
      lcd.print("*");
    }

    if(input.length()==4)
    {
```

```
delay(300);

lcd.clear();

if(input==password)
{
  lcd.print("ACCESS");
  lcd.setCursor(0,1);
  lcd.print("GRANTED");

  digitalWrite(greenLED,HIGH);

  delay(3000);

  digitalWrite(greenLED,LOW);

  attempts=0;
}
else
{
  lcd.print("ACCESS");
  lcd.setCursor(0,1);
  lcd.print("DENIED");

  digitalWrite(redLED,HIGH);

  tone(buzzer,1000);

  delay(2000);

  noTone(buzzer);

  digitalWrite(redLED,LOW);

  attempts++;

  if(attempts>=3)
```

```
      {
        lcd.clear();
        lcd.print("SYSTEM LOCK");
        lcd.setCursor(0,1);
        lcd.print("WAIT 10 SEC");

        delay(10000);

        attempts=0;
      }
    }

    input="";

    lcd.clear();
    lcd.print("ENTER PIN:");
  }
 }
}
```

Q29. Sensor Calibration and Two-Point Calibration What is Sensor Calibration?

Sensor calibration is the process of comparing a sensor's output with a known reference and adjusting its readings so that the measured values are accurate.

Why is Calibration Important in IoT?

Calibration is important because:

- It improves **measurement accuracy**.
- It reduces errors caused by manufacturing variations or environmental conditions.
- It ensures reliable data for IoT monitoring and automation.
- It provides consistent readings across different sensors.

Calibration of an Analog Soil Moisture Sensor

An analog soil moisture sensor gives raw values (typically **0–1023**) that vary from sensor to sensor. To convert these into an accurate **0–100% moisture value**, calibration is required.

Two-Point Calibration Method

The two-point calibration uses two reference measurements:

1. **Dry Point**
    - Keep the sensor completely dry (in air).
    - Record the analog reading.
    - Example: **850**
2. **Wet Point**
    - Insert the sensor into fully wet soil or water.
    - Record the analog reading.
    - Example: **350**

Then map the sensor readings between these two values:

moisture = map(sensorValue, 850, 350, 0, 100);

where:

- **0%** = completely dry soil
- **100%** = fully wet soil

This method provides much more accurate moisture percentage readings than using the default 0–1023 range.

Q30. I²C Protocol in IoT (2 Marks)

What is I²C?

I²C (Inter-Integrated Circuit) is a synchronous serial communication protocol used to connect a **microcontroller** with multiple sensors and devices using only **two communication wires**.

SDA and SCL Lines

- **SDA (Serial Data):** Transfers data between the master and slave devices.
- **SCL (Serial Clock):** Carries the clock signal generated by the master to synchronize communication.

Thus, only **two wires (SDA and SCL)** are needed, regardless of how many I²C devices are connected.

How I²C Addressing Works

- Each I²C device has a **unique address** (commonly a **7-bit address**, though 10-bit addressing also exists).
- The **master** (e.g., Arduino) sends the address of the desired slave device.

- Only the device with the matching address responds, while all other devices ignore the communication.
- This allows multiple sensors to share the same SDA and SCL lines.

Common IoT Sensors Using I²C

Examples of sensors that communicate using I²C are:

1. BMP280 – Pressure and temperature sensor
2. MPU6050 – Accelerometer and gyroscope
3. BH1750 – Light intensity sensor

These sensors can all be connected to the same I²C bus, provided each has a unique